

JARVIS

AI OPERATING SYSTEM

PRODUCTION ARCHITECTURE BLUEPRINT

Complete System Design · Step-by-Step Implementation Guide · Best Practices

10

ARCHITECTURE
LAYERS

9

BUILD PHASES

85+

API ENDPOINTS

11

AI PROVIDERS

Aliyar Solutions · Hyderabad, India · Global AI Technology Operations

Version 9.0 · May 2026 · CONFIDENTIAL

TABLE OF CONTENTS

01	Core Vision & Design Principles	3
02	Architecture Overview — 10 Layers	4
03	Layer 1 — Cloud Infrastructure (AWS)	5
04	Layer 2 — Container Orchestration	6
05	Layer 3 — Ingress, CDN & Security	7
06	Layer 4 — Backend Orchestrator (FastAPI)	8
07	Layer 5 — AI Intelligence System	9
08	Layer 6 — Memory & Data Architecture	10
09	Layer 7 — Frontend System	11
10	Layer 8 — Agent & Automation System	12
11	Layer 9 — Business Operations Integration	13
12	Layer 10 — Observability Stack	14
13	Deployment Topology Diagram	15
14	Step-by-Step Implementation Guide	16
15	Build Sequence & Priority Matrix	22
16	Technology Selection Reference	23
17	Security Architecture	24
18	Scaling Roadmap	25
19	Cost Optimisation Strategy	26

1 CORE VISION & DESIGN PRINCIPLES

What JARVIS Actually Is

JARVIS is not a chatbot. It is not an automation script. It is the central operational intelligence infrastructure of Aliyar Solutions — a full AI-native business operating system that reasons, executes, remembers, coordinates agents, manages infrastructure, and runs client-facing operations 24/7 on AWS without human intervention.

◆ Cloud Orchestration	Provisions and manages AWS infrastructure autonomously via Terraform and ECS
◆ AI Reasoning	11-provider router with circuit breakers, cost tracking, and task-type specialisation
◆ Business Automation	Outreach, CRM, proposals, invoicing, scheduling — all automated
◆ Human Identity Layer	9 named team members sign all client communications — professional and trusted
◆ Self-Healing	Circuit breakers, auto-restart, readiness probes, and incident management built in

Governing Design Principles

Operational Simplicity First	Every component must justify its existence. No complexity without a concrete operational benefit. Simplicity is not a constraint — it is a quality standard.
Progressive Enhancement	Build what works today. Add sophistication only when load or capability gaps demand it. A working system is always better than a perfectly designed unfinished one.
Infrastructure as Code Always	If it is not in Terraform, it does not exist as far as the team is concerned. No manual console clicks in production. Ever.
Secrets Never in Code	AWS Secrets Manager is the only credential store in production. Credentials are injected at runtime into ECS tasks. No .env files on servers.
Failure is Expected	Every layer assumes its dependencies will fail and handles it gracefully. Circuit breakers, retries with backoff, health probes, and auto-recovery are built-in defaults.
Observability from Day One	Logs, metrics, and health probes are not afterthoughts. They are first-class requirements from the first line of code. You cannot fix what you cannot see.

2

ARCHITECTURE OVERVIEW — 10 LAYERS

The architecture is designed as a strict separation of concerns across 10 layers. Each layer has one responsibility, one technology owner, and clear interfaces to adjacent layers. No layer bypasses another. This makes the system debuggable, replaceable, and independently scalable.

#	Layer	Core Technologies
1	Cloud Infrastructure	AWS ECS Fargate · RDS · ElastiCache · S3 · Route53
2	Container Orchestration	Docker · Docker Compose (dev) · ECS Task Definitions (prod)
3	Ingress & Edge	CloudFront · Application Load Balancer · ACM SSL · WAF
4	Backend Orchestrator	FastAPI · SQLAlchemy async · Pydantic v2 · APScheduler
5	AI Intelligence	11-provider router · Circuit breakers · Cost tracker · Task routing
6	Memory & Data	PostgreSQL · pgvector · Redis · S3 · CloudWatch Logs
7	Frontend	React 18 · Vite · Tailwind CSS · Zustand · Framer Motion
8	Agent Automation	Specialized AI agents · Task queue · Scheduled jobs · Event handlers
9	Business Operations	Gmail · Apollo · HubSpot · Slack · Stripe · Telegram · Notion
10	Observability	Prometheus · Grafana · CloudWatch · X-Ray · Alert Manager

Full System Data Flow

USER BROWSER → Route53 DNS → CloudFront (CDN + SSL termination) → Application Load Balancer → [Frontend: React/Nginx] or [Backend: FastAPI] → AI Intelligence Router → Memory Layer (PostgreSQL + Redis) → Agent & Automation System → Business Operations APIs

All compute runs in private subnets. No ECS task has a public IP address. Traffic enters only through the ALB. Outbound AI API calls route through a NAT Gateway. This is the correct zero-trust production posture.

3

LAYER 1 — CLOUD INFRASTRUCTURE

The cloud foundation is built on AWS ap-south-2 (Hyderabad) as primary and ap-south-1 (Mumbai) as disaster recovery. All infrastructure is defined in Terraform and lives in the infra/terraform/ directory. Nothing is provisioned manually.

Why This Technology

ECS Fargate is chosen over raw EC2 because it eliminates all server management overhead: no OS patching, no capacity planning, no instance health management. You define a task and AWS runs it. Auto-scaling adds tasks when CPU exceeds 70%, removes them when load drops. For a 24/7 autonomous system, this is the only defensible choice.

Components

Component	Purpose & Configuration
VPC	Private subnets for compute, public subnets for ALB only. NAT Gateway for outbound.
ECS Fargate	Serverless container runtime. Backend: 0.5 vCPU / 1GB. Frontend: 0.25 vCPU / 512MB.
RDS PostgreSQL Multi-AZ	Primary in ap-south-2a, standby in ap-south-2b. Automatic failover <60s.
ElastiCache Redis	Single-node t3.micro for dev, cluster mode for production scale.
S3	Proposals, invoices, attachments, frontend static assets (CloudFront origin).
Secrets Manager	All API keys, DB passwords, webhook tokens. Injected into ECS at runtime.
Route53	DNS management. A records pointing to CloudFront distribution.
ACM	Free SSL/TLS certificates. Auto-renewed. Attached to CloudFront and ALB.

4

LAYER 2 — CONTAINER ORCHESTRATION

Three environments use the same Docker images but different orchestration strategies. This ensures what works locally behaves identically in production.

Why This Technology

Docker Compose is sufficient for local development and staging validation. ECS task definitions provide the same container configuration in production. The critical rule: build images once, promote the same image from dev to staging to production. Never rebuild in CI for production — tag by git SHA, promote by SHA.

Components

Component	Purpose & Configuration
Development	docker-compose.yml — postgres, redis, backend, frontend, nginx. All with health checks.
Production images	Multi-stage Dockerfile. Build stage installs dependencies, runtime stage is minimal.
Image registry	AWS ECR. Images tagged by git SHA. Latest tag also updated on main branch deploys.
Task definitions	JSON in infra/terraform/. CPU, memory, port mappings, environment, secrets ARNs.
Service auto-scaling	Target tracking — 70% CPU threshold. Min 1 task, max 4 tasks per service.
Deployment strategy	Blue/Green rolling. New tasks start, health checks pass, old tasks drain and stop.

5

LAYER 3 — INGRESS, CDN & SECURITY

All traffic enters through a single controlled path. No service is directly internet-accessible. This layer handles SSL termination, caching, routing, and the first line of security defence.

Why This Technology

CloudFront in front of everything — not just static files — provides DDoS protection at the edge, SSL termination closest to the user, and caches API responses that are safe to cache. The ALB handles path-based routing: `/api/*` goes to backend, everything else goes to the React frontend served by Nginx.

Components

Component	Purpose & Configuration
CloudFront	Global CDN. Caches React static assets. Forces HTTPS. Custom domain with ACM cert.
ALB	Layer 7 load balancer. Path routing. Health checks on <code>/health</code> . Sticky sessions for WebSocket.
WAF	Block SQL injection, XSS, common scanners. Rate limit by IP (add after launch).
Security Groups	Backend SG: only accepts traffic from ALB SG. DB SG: only accepts from backend SG.
HTTPS enforcement	CloudFront redirects HTTP → HTTPS. HSTS headers. TLS 1.2 minimum.
Cache invalidation	CloudFront paths invalidated on every frontend deployment via <code>deploy.yml</code> .

6 LAYER 4 — BACKEND ORCHESTRATOR

FastAPI is the operational core of JARVIS. Every business action, AI call, agent dispatch, and data operation flows through it. It is asynchronous throughout — no blocking I/O anywhere in the call stack.

Why This Technology

FastAPI with SQLAlchemy 2.0 async and asyncpg achieves ~10,000 requests/second on a single 0.5 vCPU container for non-AI endpoints. AI endpoints are network-bound (waiting for OpenAI/Claude), so async is critical — the event loop handles thousands of concurrent AI requests without blocking.

Components

Component	Purpose & Configuration
19 route modules	chat, briefing, approvals, agents, crm, leads, outreach, tasks, intelligence, governance, emergency, knowledge, catalog, ai_ops, team, auth, scheduler, calendar, sync.
Request middleware	X-Request-ID on every request. X-Response-Time in every response. Structured JSON logs.
Error handling	Global exception handlers. Standardised {error, status, path, request_id} envelope.
Connection pooling	pool_size=10, max_overflow=20. Add PgBouncer when scaling beyond 2 containers.
Auto-seed on boot	Service catalog and team registry seed themselves on first startup. No manual steps.
/health + /readyz	Liveness probe (no DB) and readiness probe (DB ping, AI, scheduler, team registry).

7

LAYER 5 — AI INTELLIGENCE SYSTEM

The AI router is the primary competitive differentiator of JARVIS. No SaaS product or off-the-shelf tool provides this level of multi-provider orchestration, cost awareness, and automatic failover in a single system.

Why This Technology

Single-provider AI systems fail when that provider has an outage, rate limits your account, or raises prices. The 11-provider circuit breaker architecture means JARVIS never goes down due to an AI provider failure — it simply routes to the next available provider in the fallback chain, transparently and instantly.

Components

Component	Purpose & Configuration
11 providers	Anthropic, OpenAI, Google, DeepSeek, Groq, Mistral, Moonshot, ZhipuAI, Qwen, MiniMax, NVIDIA.
Task-type routing	CODE→Claude Sonnet, REASONING→Claude Opus, RESEARCH→Gemini, FAST→DeepSeek, REALTIME→Groq.
Circuit breakers	3 failures → OPEN state → 5-minute cooldown → HALF_OPEN probe → CLOSED on success.
Cost tracking	Per-call USD estimate from 18-entry rate table. Daily aggregation. \$50 surge alert.
AI response caching	NEXT: Hash prompt+context → check Redis → return cached if found. 20-40% cost reduction.
Audit log	Every AI call persisted: provider, model, tokens, latency_ms, cost_usd, success.

8

LAYER 6 — MEMORY & DATA ARCHITECTURE

JARVIS's intelligence depends on its memory. A system that cannot remember is a system that cannot learn. The data architecture is designed to support operational storage, intelligent retrieval, and high-speed caching simultaneously.

Why This Technology

PostgreSQL handles all relational data. pgvector extends it with vector similarity search — turning the database into a semantic memory store. Redis handles everything that needs sub-millisecond access: sessions, rate limits, AI response cache, real-time counters. S3 handles unstructured binary data: PDFs, images, exports.

Components

Component	Purpose & Configuration
PostgreSQL (RDS)	All operational data. 15 SQLAlchemy models. Multi-AZ. Daily automated backups.
pgvector	NEXT: Add to RDS. Generate embeddings for memories, proposals, conversations. Semantic recall.
Redis (ElastiCache)	Sessions, AI response cache (TTL 1h), rate limit counters, real-time pub/sub for WS.
S3	Proposal PDFs, invoice documents, exported reports, frontend build artifacts.
CloudWatch Logs	All container stdout/stderr. 90-day retention. Queryable with CloudWatch Insights.
Named volumes (dev)	postgres_data, redis_data — persist across container restarts in local dev.

13

[illegible]

Environment	Compute	Database	Cache	Purpose
Development	Laptop (uvicorn --reload)	Docker PostgreSQL	Docker Redis	Feature development
Local Staging	docker-compose up	Docker PostgreSQL	Docker Redis	Integration testing
Production	ECS Fargate (Hyderabad)	RDS Multi-AZ	ElastiCache	24/7 live system
DR Standby	ECS (Mumbai)	RDS Read Replica	ElastiCache	Failover target

14

STEP-BY-STEP IMPLEMENTATION GUIDE

This guide walks through every implementation step from a blank AWS account to a fully operational JARVIS system. Each step includes the exact commands, expected outputs, and validation checks. Follow the steps in order — each builds on the previous.

PHASE 1 — AWS FOUNDATION

Create AWS Account & Configure IAM

- Go to aws.amazon.com → Create account (use business email)
- Enable MFA on root account immediately — this is non-negotiable
- Create IAM user 'jarvis-deploy' with programmatic access
- Attach policies: AmazonECS_FullAccess, AmazonEC2FullAccess, AmazonRDSFullAccess,
- AmazonElastiCacheFullAccess, AmazonS3FullAccess, SecretsManagerReadWrite
- Download and save the access key CSV — you will not see it again
- Install AWS CLI: `pip install awscli`
- Configure: `aws configure` (enter key, secret, region: ap-south-2, format: json)
- Validate: `aws sts get-caller-identity` → should return your account ID

Install & Configure Terraform

- Download Terraform 1.9+ from terraform.io/downloads
- Add to PATH: `sudo mv terraform /usr/local/bin/`
- Validate: `terraform version` → should show 1.9.x
- Navigate to `infra/terraform/` directory
- Create `terraform.tfvars` with your specific values (never commit this file)
- Run: `terraform init` → downloads AWS provider
- Run: `terraform plan` → shows what will be created (no changes made yet)
- Review the plan output — verify VPC, subnets, ECS cluster, RDS are listed

Provision Core Infrastructure

- Run: `terraform apply` → type 'yes' when prompted
- Wait 8–12 minutes for RDS to provision (this is the slowest component)
- Expected outputs: `vpc_id`, `rds_endpoint`, `ecs_cluster_arn`, `alb_dns_name`
- Save the outputs — you'll need them for environment configuration
- Verify in AWS Console: ECS cluster 'jarvis-production-cluster' exists
- Verify: RDS instance is 'Available' status in ap-south-2a
- Verify: ElastiCache cluster is 'Available'
- Verify: S3 bucket 'jarvis-production-assets' exists

Configure AWS Secrets Manager

- In AWS Console → Secrets Manager → Store a new secret
- Create secret 'jarvis/production/env' with key-value pairs:
- ANTHROPIC_API_KEY = sk-ant-...
- OPENAI_API_KEY = sk-...
- GOOGLE_API_KEY = Alza...
- SLACK_WEBHOOK_URL = https://hooks.slack.com/...
- TELEGRAM_BOT_TOKEN = ...
- (all other API keys)
- Note the secret ARN — add it to your ECS task definition in Terraform
- Test access: `aws secretsmanager get-secret-value --secret-id jarvis/production/env`

PHASE 2 — CONTAINER BUILD & REGISTRY

5

Create ECR Repositories & Build Images

- Create ECR repos: `aws ecr create-repository --repository-name jarvis-production-backend`
- Create ECR repos: `aws ecr create-repository --repository-name jarvis-production-frontend`
- Get ECR login token: `aws ecr get-login-password | docker login --username AWS`
- `--password-stdin .dkr.ecr.ap-south-2.amazonaws.com`
- Build backend: `docker build -t jarvis-backend ./backend`
- Tag: `docker tag jarvis-backend:latest /jarvis-production-backend:latest`
- Push: `docker push /jarvis-production-backend:latest`
- Repeat build/tag/push for frontend (./frontend directory)
- Validate: both images appear in ECR console with 'latest' tag

6

Configure ECS Task Definitions

- ECS task definitions are in `infra/terraform/ecs.tf` — review and update image URIs
- Ensure the task definition includes `secretsFrom` pointing to your Secrets Manager ARN
- Backend task: 512 CPU units (0.5 vCPU), 1024 MB memory
- Frontend task: 256 CPU units (0.25 vCPU), 512 MB memory
- Set log configuration: `awslogs` driver, log group `/jarvis/backend`, region `ap-south-2`
- Run `terraform apply` to register the new task definitions
- Validate: ECS console shows task definition revisions for both services

PHASE 3 — NETWORKING & SSL

7

Domain Configuration & SSL Certificate

- Register domain or transfer to Route53 (if not already there)
- In Route53: create hosted zone for your domain
- In ACM (us-east-1 region — required for CloudFront): request public certificate
- Add domain: aliyarsolutions.com and *.aliyarsolutions.com
- Validate via DNS: ACM creates CNAME records — add them to Route53
- Wait for certificate status to show 'Issued' (5–15 minutes)
- IMPORTANT: ACM certificate for CloudFront MUST be in us-east-1 region

8

Configure CloudFront Distribution

- Create CloudFront distribution with ALB as origin
- Set Origin Protocol: HTTPS only (ALB listens on 443)
- Cache policy: Managed-CachingDisabled for /api/* paths (API responses must not cache)
- Cache policy: Managed-CachingOptimized for /* (static assets — long TTL)
- Viewer Protocol: Redirect HTTP to HTTPS
- Attach ACM certificate from us-east-1
- Set CNAME: jarvis.aliyarsolutions.com
- In Route53: Create A record (alias) pointing to CloudFront distribution domain
- Wait for CloudFront deployment (15–20 minutes for global propagation)
- Validate: `https://jarvis.aliyarsolutions.com/health` → {status: ok}

9

Configure ALB & Target Groups

- ALB listener on port 443 with ACM certificate (separate from CloudFront cert — use ap-south-2)
- Create listener rule: if path starts with /api/ → forward to backend target group
- Default rule: forward to frontend target group
- Backend target group: health check path /health, port 8000, threshold 2/2
- Frontend target group: health check path /, port 80, threshold 2/2
- Enable WebSocket: set ALB idle timeout to 300 seconds in load balancer attributes
- Add stickiness to backend target group (needed for WebSocket connection persistence)

PHASE 4 — DATABASE & CACHE SETUP

10

Initialize PostgreSQL Database

- Connect to RDS from a bastion EC2 or via AWS Session Manager (no SSH keys needed)
- Alternative: temporarily add your IP to the RDS security group for setup only
- Connect: `psql -h -U jarvis -d jarvis_db`
- JARVIS auto-creates all tables on startup via SQLAlchemy `init_db()`
- No manual schema creation required — all models are registered in `app/models/__init__.py`
- Run `infrastructure/postgres/init.sql` for any custom PostgreSQL extensions
- Validate: after first JARVIS startup, run `\dt` to see all 15+ tables created
- Verify auto-seed: `SELECT count(*) FROM service_divisions;` → should return 30
- Verify auto-seed: `SELECT count(*) FROM team_members;` → should return 9

11

Configure Redis & Test Connectivity

- ElastiCache Redis is provisioned by Terraform — no manual setup needed
- Get the Redis endpoint from terraform output or ElastiCache console
- Backend receives `REDIS_URL` via environment: `redis://:6379/0`
- Test from backend container: `redis-cli -h ping` → PONG
- Validate session storage: make a chat request, check Redis keys: `KEYS *`
- Redis is used for: WebSocket pub/sub, AI response cache, rate limit counters

PHASE 5 — CI/CD PIPELINE

12

Configure GitHub Actions Secrets

- In GitHub repository → Settings → Secrets and Variables → Actions
- Add secrets (these are NEVER stored in code):
- AWS_ACCESS_KEY_ID — from IAM user jarvis-deploy
- AWS_SECRET_ACCESS_KEY — from IAM user jarvis-deploy
- SLACK_WEBHOOK_URL — for deployment notifications
- APP_BASE_URL — <https://jarvis.aliyarsolutions.com>
- Verify: the deploy.yml workflow is in .github/workflows/
- The workflow triggers on: push to main branch, or manual workflow_dispatch

13

First Production Deployment

- Merge your branch to main (or trigger manually via GitHub Actions UI)
- Watch the Actions tab — build job starts: builds backend + frontend Docker images
- Images are tagged with git SHA and pushed to ECR
- Deploy job: downloads current ECS task definition, updates image, registers new revision
- ECS performs rolling blue/green deployment — new task starts before old task stops
- Wait for ECS service to reach 'steady state' (wait-for-service-stability: true)
- Post-deploy: workflow hits /health and /readyz — pipeline fails if not 200
- Slack notification: ■ JARVIS deployed + health-validated
- Validate: <https://jarvis.aliyarsolutions.com> → JARVIS dashboard loads

14

Validate Full System Health

- Run: `curl https://jarvis.aliyarsolutions.com/health`
- Expected: {status: ok, system: JARVIS, version: 9.0.0}
- Run: `curl https://jarvis.aliyarsolutions.com/readyz`
- Expected: {status: ready, checks: {database: {status: ok}, ai_providers: {available: N}}}
- Open dashboard: verify all 18 views load without errors
- POST /api/v1/catalog/seed and /api/v1/team/seed (auto-seeded on boot, but verify)
- Test chat: send a message, verify AI response with correct provider routing
- Check AI Ops dashboard: verify circuit breakers are all CLOSED
- Run: make readyz from development machine → should show all green

PHASE 6 — MONITORING & OBSERVABILITY

15

Configure CloudWatch Logs & Alarms

- ECS Fargate automatically sends container logs to CloudWatch (awslogs driver in task def)
- Log groups created: /jarvis/backend, /jarvis/frontend
- Set retention: 30 days (cost control — adjust to 90 days if budget allows)
- Create CloudWatch Metric Filter for ERROR log pattern → triggers alarm
- Create Alarm: 5xx error rate > 1% over 5 minutes → SNS → Slack notification
- Create Alarm: ECS CPU > 80% sustained 5 minutes → scale out trigger
- Create Alarm: RDS connections > 80% of max → alert (add connection pooler)
- Dashboard: create CloudWatch dashboard with key metrics for quick operational view

16

Deploy Grafana & Prometheus

- Deploy Grafana as ECS task (internal ALB only — not public-facing)
- Or use Grafana Cloud free tier (recommended — zero operational overhead)
- Connect Prometheus remote_write to Grafana Cloud Prometheus endpoint
- Import JARVIS dashboard JSON from monitoring/grafana/dashboard.json
- Configure alert notification channel: Slack webhook
- Set up 7 alert rules from monitoring/alert_rules.yml
- Test alerts: manually stop a container, verify alert fires within 2 minutes

15

BUILD SEQUENCE & PRIORITY MATRIX

Priority	What to Build	Why Now	Time Estimate	Impact
1 — NOW	Ship to ECS via existing Terraform + deploy.yml	Foundation everything else depends on	2–4 hours	Critical
2 — NOW	Route53 + ACM + CloudFront + ALB on real domain	Makes system publicly accessible with HTTPS	2 hours	Critical
3 — NEXT	Redis semantic AI response caching	20–40% immediate AI cost reduction	4 hours	High ROI
4 — NEXT	pgvector + embedding memory layer	Transforms keyword search into semantic recall	6 hours	High
5 — WEEK 1	Webhook receivers (HubSpot, Gmail, Apollo)	Real-time data vs polling. Removes cron debt.	6 hours	High
6 — WEEK 2	Redis-backed persistent task queue	Needed only when running 2+ backend containers	4 hours	Medium
7 — WEEK 2	AWS X-Ray distributed tracing	Pinpoints performance bottlenecks with real traffic	3 hours	Medium
8 — MONTH 1	WAF rules + rate limiting per endpoint	Security hardening after system is stable	3 hours	Medium
9 — MONTH 2	Multi-region DR (Mumbai failover)	Business continuity. Only after primary is stable.	8 hours	Long-term
10 — FUTURE	EKS migration from ECS Fargate	Only if ECS cannot handle load (unlikely soon)	40+ hours	Low now

16

TECHNOLOGY SELECTION REFERENCE

Why Each Technology Was Chosen Over Alternatives

Decision	Chosen	Alternatives Considered	Reason for Choice
Container runtime	ECS Fargate	EC2, EKS, Lambda	No server management. Auto-scaling. Native AWS integration. EKS is overkill at this scale.
Database	PostgreSQL (RDS)	MySQL, DynamoDB, MongoDB	ACID compliance. JSON columns. Future pgvector. Relational data model fits business data perfectly.
Cache/Queue	Redis	Memcached, SQS, RabbitMQ	Multi-purpose: cache + pub/sub + queue. Single dependency instead of three. Native TTL support.
AI Framework	Custom router	LangChain, LlamaIndex	LangChain adds abstraction cost, debugging complexity, and version instability. Direct SDK calls are simpler, faster, and fully controllable.
Backend	FastAPI	Django, Express, Flask	Async-native, high performance (~10k req/s), auto-generated OpenAPI docs, Pydantic validation built-in.
Frontend	React + Vite	Next.js, Vue, Angular	Vite build is 10× faster than CRA. React ecosystem. No SSR needed for an internal ops dashboard.
IaC	Terraform	CDK, CloudFormation, Pulumi	Provider-agnostic, mature ecosystem, most community resources, readable HCL syntax.
Reverse proxy	Nginx	Caddy, Traefik, HAProxy	Industry standard. Zero configuration for static file serving. Mature, predictable, minimal.
Scheduling	APScheduler	Celery, Airflow, AWS EventBridge	Zero additional infrastructure. Persistent jobs via SQLAlchemy. Sufficient for 7 cron jobs.
Secret storage	AWS Secrets Manager	.env files, SSM Parameter Store	Encrypted, audited, version-controlled, native ECS integration. SSM is cheaper but less featured.

17

SECURITY ARCHITECTURE

Zero-Trust Network Security

Every component operates on the principle of least privilege. No service can communicate with another unless an explicit security group rule permits it. No credentials exist in any codebase, Docker image, or environment file on any server.

Security Layer	Controls	Threat Mitigated
Network (VPC)	Private subnets, NAT Gateway, security groups	Direct server access, lateral movement
Edge (CloudFront + WAF)	DDoS protection, bot filtering, geo-blocking	Volume attacks, scrapers, injection
Transport (ACM)	TLS 1.2+ enforced, HSTS headers	MITM, traffic interception
Authentication (ALB)	HTTPS-only listener, certificate pinning	Downgrade attacks
Secrets (Secrets Manager)	KMS-encrypted, rotation-capable, IAM-controlled	Credential leakage
Container (ECS)	Non-root user, read-only filesystem, no SSH	Container escape, privilege escalation
Code (gitleaks)	Pre-commit secret scanning, 12 pattern types	Accidental secret commits
API (middleware)	Request IDs, rate limits, standardised errors	Enumeration, replay attacks
Database (RDS)	Private subnet, encrypted at rest, IAM auth option	Unauthorised data access
Dependencies	Pin all versions in requirements.txt	Supply chain attacks

CRITICAL RULE: If a secret is ever accidentally committed to git, treat it as compromised immediately. Rotate it before doing anything else. Then investigate how it happened and add a gitleaks rule to prevent it. A committed secret is a breached secret.

18

SCALING ROADMAP

Traffic Level	Monthly Requests	Architecture Change	Expected Cost
Stage 0 — Launch	< 10k / month	1 ECS task each. Single-AZ Redis.	~\$80–120 / month
Stage 1 — Growth	10k–100k / month	Min 2 ECS tasks. Enable auto-scaling. Multi-AZ Redis.	~\$200–350 / month
Stage 2 — Scale	100k–1M / month	Add PgBouncer. Redis cluster. CloudFront edge caching.	~\$500–800 / month
Stage 3 — Enterprise	1M–10M / month	ECS capacity providers. Read replicas for RDS. CDN aggressive caching.	~\$1,500–3,000 / month
Stage 4 — Platform	> 10M / month	Evaluate EKS. Multi-region active-active. Aurora Serverless.	Custom quote

The current architecture comfortably handles Stage 0 through Stage 2 without any architectural changes — only configuration changes (more tasks, bigger instances). The first real architectural evolution happens at Stage 3. You will know when you need it because your monitoring will tell you before it becomes a problem.

19

COST OPTIMISATION STRATEGY

Infrastructure Cost Control

Cost Area	Current Approach	Optimisation	Saving Potential
AI API calls	All calls hit provider APIs	Redis semantic cache (TTL 1h)	20–40% reduction
ECS Fargate	On-demand pricing	Fargate Spot for non-critical tasks	50–70% on eligible tasks
RDS	db.t3.micro initially	Reserved instance (1-year)	30–40% vs on-demand
Data transfer	All via NAT Gateway	VPC endpoints for S3/ECR	\$0.045 → \$0 per GB
CloudWatch Logs	Default retention	Set 30-day retention on all log groups	70% log storage saving
ECR storage	All image versions	Lifecycle policy: keep last 10 images only	Negligible but clean
AI model selection	Expensive models for all tasks	Task-type routing (FAST→DeepSeek)	60–80% on FAST tasks

Monthly Cost Estimate at Launch

Service	Specification	Estimated Monthly Cost
ECS Fargate — Backend	0.5 vCPU / 1GB / 730 hours	~\$15
ECS Fargate — Frontend	0.25 vCPU / 512MB / 730 hours	~\$8
RDS PostgreSQL	db.t3.micro, Multi-AZ, 20GB storage	~\$30
ElastiCache Redis	cache.t3.micro, single node	~\$12
ALB	730 hours + LCU charges	~\$18
CloudFront	First 1TB data transfer free	~\$0–5
NAT Gateway	730 hours + data processing	~\$35
S3	10GB storage + requests	~\$2
Secrets Manager	5 secrets × \$0.40	~\$2
CloudWatch	Logs + metrics + alarms	~\$5
Route53	1 hosted zone + queries	~\$1
TOTAL		~\$130–140 / month

The \$130–140/month baseline gives you a production-grade, 24/7 autonomous AI operating system serving real clients globally. This is less than a single mid-level developer's daily salary. At Stage 1 growth, the system earns multiples of its infrastructure cost through automated outreach, proposals, and client operations.

The foundation is built. The architecture is sound. The path is clear.

Ship to production. Validate the system is alive. Then build the next layer on top of a known-good foundation. Everything in this document exists to serve one mission: making Aliyar Solutions the most operationally efficient AI services company in the world.

Aliyar Solutions · JARVIS v9.0.0 · May 2026 · CONFIDENTIAL